

SmartEgg Grader: A Weight Sensor-Based IoT Egg Sorting System with Voice Features

Rod Geryk C. Navarro
*School of Electrical, Electronics, and
Computer Engineering
Mapua University
Manila, Philippines
rgcnavarro@mymail.mapua.edu.ph*

Abstract — In Manual egg grading often leads to human errors that compromise the consistency and quality of egg classification, especially in small poultry farms that cannot afford expensive commercial-grade machines. This study introduces a low-cost, IoT-based smart egg grading system that uses egg weight as a key parameter for classification, ensuring standardization and improving efficiency. The prototype integrates an Arduino Uno R3 as the main controller and an ESP32 module for wireless cloud monitoring via ThingSpeak. It features two modes, home and automatic, with a rotary encoder for mode switching, a load cell for weight measurement, servo motors for egg handling, and dual LCD screens for real-time feedback. The system also includes voice prompts using a DFPlayer Mini to guide users through the process. Eggs are categorized into Grade A (28–55g) and Grade B (20–27g), with data on egg count, weight, and sorting duration uploaded to the cloud. The system was tested using plastic balls simulating real egg weights and successfully demonstrated accurate classification, responsive feedback, and reliable cloud integration. This smart grader offers an accessible, automated, and cloud-connected solution ideal for small-scale poultry operations.

Keywords — *Grading System, Arduino Uno, Weight Sensor, LCD, DF Player Mini*

I. INTRODUCTION

Human error can occur during the manual grading of egg quality by farmers. This situation may result in the misidentification of high-quality eggs versus those of inferior quality [1]. The main purpose of an egg grading system is to categorize eggs based on quality, size, and weight, ensuring consumers receive eggs that meet specific safety and quality standards. The system helps maintain consumer trust, facilitates efficient marketing, and guarantees a uniform quality of eggs for both consumers and producers. To address this problem an egg grading system is used. Egg weight is a key factor in determining quality and meeting market standards. It plays a crucial role in helping farmers sort and sell eggs that match the specific requirements of their target market. [2]. Many poultry farms are forced to rely on manual egg sorting because modern grading machines are too expensive. Despite the high cost, these machines offer important features that farmers need to improve their operations, features they simply cannot afford, leaving them at a disadvantage. [3]. This study, therefore, introduces a cost-effective system that leverages Arduino Uno R3 as the main microcontroller, and ESP32 Development module as a WiFi module of the system for standardization and cloud monitoring

of eggs based on their weight suitable for many small poultry farms. This study focuses on creating a prototype of an egg grading system that has an IoT feature that helps farmers to monitor the grading process effectively such as the number of eggs sorted, their sizes, the time of sorting, and sorting duration, for remote monitoring and performance analysis.

The system operates in two screen modes: home screen and automatic screen mode. In home screen mode, in this screen the system shows the total number of eggs sorted, and the time duration of the last sort. Initially, this is the first screen that will show at the start of the system, and no grading will happen at this screen, whenever the user goes to this mode the system will deactivate the weight sensor and servo motors. While in the automatic mode screen it activates the automation of the grading process, weight sensor and the servo motors will be turned ON to begin the grading process, it also shows the weight in grams and kilo grams of the object placed above the loadcell. Two LCD screens provides the output information of the sorted eggs, ensuring clear feedback on the system's performance. The first LCD screen allows that user to see the two modes with information such as total number of eggs sorted, time duration of the last sort, and the current weight of the egg placed on the loadcell. The second screen, shows the two grades the grade A and grade B that helps distinguish which egg has an optimal weight and standard weight. By integrating embedded systems with automation, this project highlights a sustainable approach to small-scale egg grading system while bridging the gap between industrial egg sorting techniques and cost-effective innovations.

II. METHODOLOGY

The development of the smart egg sorting system involves assembling and integrating various hardware components to ensure smooth and efficient operation. The Arduino Uno R3 serves as the central controller, managing inputs from the weight sensor with a capacity of 1kg of load, rotary encoder that allows the user to change modes, and DF player mini that reads data from the SD card in order for the system to output audio. The grading process starts with the user rotating the rotary encoder clockwise to start the grading, in the automatic mode the system will trigger the servo1 to rotate 180 degrees to retrieve the egg from the egg automatic roller one by one to deliver it to the weight sensor platform to detect its weight. After the detection of the weight, servo2 will rotate the whole platform to the right or left depending on the weight of the egg to separate it. If the final weight is in between 28 and 55 grams the servo2 will rotate

60 degrees to route the egg to its corresponding place. On the other hand, if the final weight is in between 20 to 27 grams servo2 will rotate to an angle of 130 degrees to sort the egg to its right place. If there is no egg detected on the platform the system will automatically return to home screen to display the sorted eggs and last sort duration on the first LCD.

In automatic mode, the system will begin grading from retrieving the egg from the egg automatic roller to the classifying the eggs based on its weight. With the use of the ESP32 as the WiFi module of the system it allows the device to connect to the internet to monitor the data of the grading process. After each sort it will update the latest data to ThingSpeak platform. This system includes a voice feature the guides the user thorough out the system process like informing the user if the device starts the grading process, the information about the graded egg categorize class, and informing the user when the system is updating to cloud. An LCD continuously updates the total count of eggs and eggs per weight, providing real-time feedback. The entire system is powered through a regulated power source specifically a four 18650 lithium battery to ensure consistent performance.

A. Hardware Block Diagram

The hardware block diagram of the egg grading system shows the connection and function of each component used in the prototype. At the center is the Arduino Uno R3, which acts as the main controller and is responsible for handling data from input devices and sending commands to output devices. The HX711 module, connected to a 1kg load cell, serves as the weight sensor that sends egg weight data to the Arduino. A rotary encoder is used as the mode selector, allowing the user to switch between the home screen and automatic grading mode. For visual output, the system uses two 16x2 I2C LCD displays, one for showing system information like egg weight, mode, and total count, and the other for displaying the classification result (Grade A or B). Two servo motors are used: Servo 1 moves eggs from the roller to the weighing platform, and Servo 2 tilts the platform to sort the egg based on its weight. The ESP32 module is connected to the Arduino via serial communication and enables WiFi capability, allowing the system to upload sorting data to the ThingSpeak cloud platform. A DFPlayer Mini with a speaker is included for voice prompts, helping guide the user through each step of the process. All these components are powered by a battery pack of four 18650 lithium-ion batteries, regulated to supply stable voltage to the entire system.

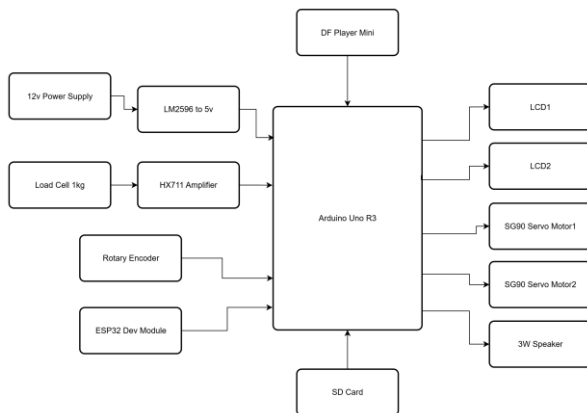


Fig. 1. Block Diagram of the Smart Egg Grading System

B. System Flowchart

Fig. 2 represents the proposed system's flowchart. When the system is powered on, it initializes all the components including the Arduino Uno, LCD screens, load cell (weight sensor), servo motors, DFPlayer Mini (audio), and rotary encoder. Once initialized, the system displays the Home Screen, which shows the total number of eggs sorted and the duration of the last grading session. In this mode, all sorting functions are disabled, and the system waits for user input. The user interacts with the system using a rotary encoder. If the encoder is turned clockwise, the system switches to Automatic Mode. At this point, the first servo motor (Servo1) becomes active and retrieves one egg from the automatic roller. It carefully places the egg on the weighing platform that contains the load cell sensor. The system then reads and calculates the egg's weight in grams and kilograms.

Once the weight is measured, the second servo motor (Servo2) is triggered. If the egg weighs between 28g to 55g, it is considered Grade A, and Servo2 rotates to 60° to sort the egg into the correct section. If the egg weighs between 20g to 27g, it is considered Grade B, and the servo rotates to 130° for sorting. Any egg below 20g is rejected as too light, and the system returns to Home Mode. After sorting, the system increments the egg counter and plays an audio message to confirm the classification using the DFPlayer Mini. At the same time, it sends updated data, such as the total eggs sorted, Grade A and B counts, and sorting duration to the ESP32 module, which uploads the data to the ThingSpeak cloud platform for remote monitoring. The system continues this loop until the user turns the rotary encoder clockwise, which brings the system back to the Home Screen. This switch also disables the servos and weight sensor. In both modes, the LCD screens continuously show real-time updates about egg count, classification, and weight.

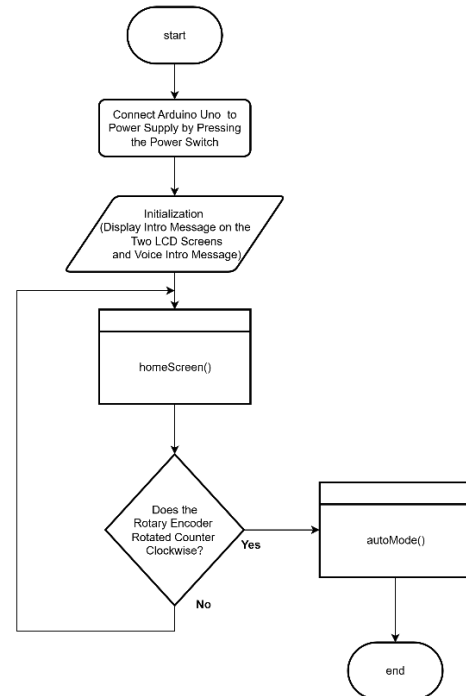


Fig. 2. The Main Flowchart of the System

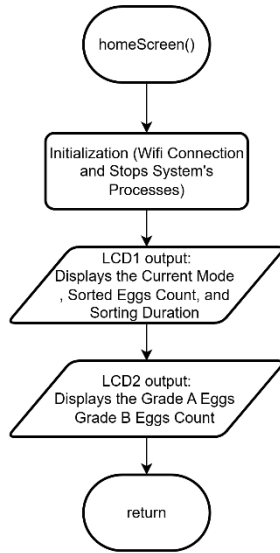


Fig. 3. Home Screen Function

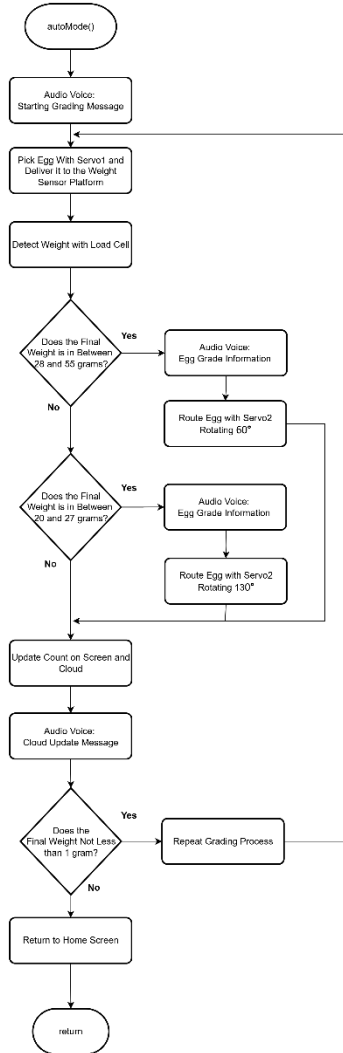


Fig. 4. Automatic Mode Function

C. Testing

During the testing of the prototype, real eggs were not used due to their fragile nature and the current state of the system, which still had parts that could potentially damage actual eggs. To safely test the sorting and weighing functions, plastic toy balls were used as a temporary substitute. These balls were carefully chosen to closely match the shape and weight of real eggs, ensuring a realistic testing environment without the risk of cracks or breakage. Specifically, two toy balls were prepared with a weight of approximately 37 grams (to represent Grade A eggs), and another two weighed around 24 grams (to simulate Grade B eggs). The system successfully identified and sorted these based on their weight range, with the servo motors rotating to the appropriate angles as programmed. The LCD screens accurately displayed the weight of each ball in both grams and kilograms, while the DFPlayer Mini provided audio feedback after every sort. After each classification, data such as total eggs sorted and sorting duration were successfully sent to the ThingSpeak platform using the ESP32 module. This testing method proved effective in validating the core functionalities of the system while protecting the physical integrity of the hardware and ensuring the safety of future real-egg tests.



Fig. 5. SmartEgg Grader Front View

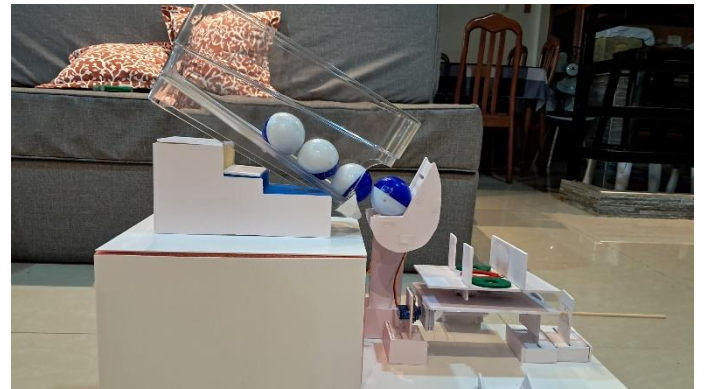


Fig. 6. Sliding of the Eggs in the Egg Roller



Fig. 7. Process of Getting the Eggs One by One from the Roller to the Weight Sensor Platform



Fig. 11. SmartEgg Grader with the Circuit



Fig. 8. SmartEgg Grader Side View

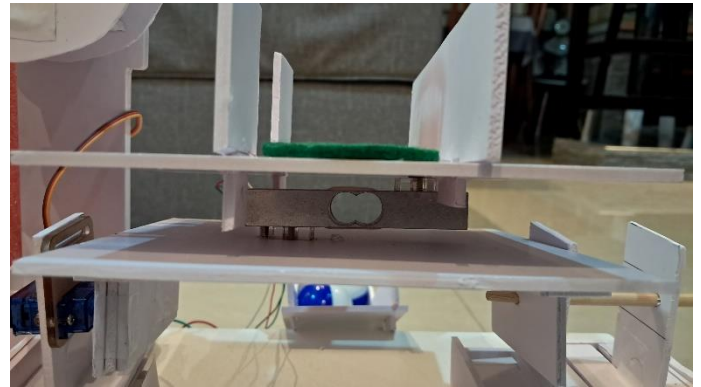


Fig. 12. SmartEgg Grader Weight Sensor Platform

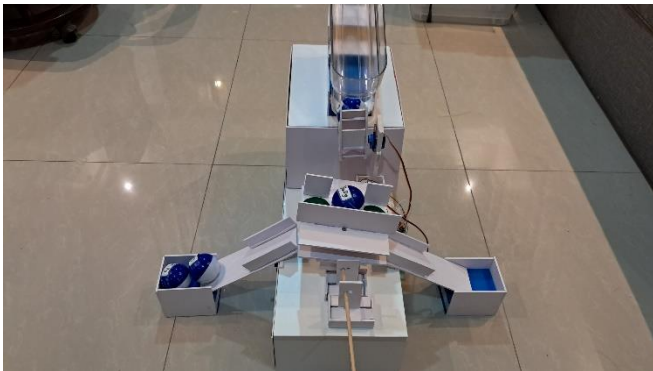


Fig. 9. SmartEgg Grader Two Routers, One for Grade A and One for Grade B Eggs



Fig. 10. SmartEgg Grader Two LCD Screens

For Fig. 16-18, For Fig. 13-15, this Arduino code controls an egg grading system that uses a weight sensor (HX711), servo motors, an LCD display, a rotary encoder, and a DFPlayer Mini for audio. It switches between home and automatic modes using a rotary encoder and sorts eggs based on their weight into Grade A or B using servo motors. The LCDs display sorting data like weight and egg counts, while the ESP32 (connected via serial) uploads results to the cloud. The DFPlayer provides voice feedback throughout the process to guide the user.

```

NavarroRGC_FinalProject_CPE162P.ino
1 #include <Servo.h>
2 #include <HX711_ADC.h>
3 #include <DFPlayer.h>
4 #include <Encoder.h>
5 #include <SoftwareSerial.h>
6 #include <DFRobotDFPlayerMini.h>
7
8 //C:\Users\User\AppData\Local\Arduino15\libraries
9
10
11 static const uint8_t PIN_MP3_TX = 12;
12 static const uint8_t PIN_MP3_RX = 11;
13
14 // Global variables
15 unsigned long autoStartTime = 0;
16 float autoDurationMinutes = 0.0;
17 bool autoModeStarted = false;
18
19 unsigned long sortStartTime = 0;
20
21 unsigned long lastUpdateTime = 0;
22 const unsigned long homeUpdateInterval = 500; // Update every 500 ms
23
24 SoftwareSerial softwareSerial(PIN_MP3_RX, PIN_MP3_TX);
25
26 DFRobotDFPlayerMini player;
27
28 // pin assignments
29 #define SERVO1_PIN 7
30 #define SERVO2_PIN 5
31 #define LONDCELL_DT 2
32 #define LONDCELL_SCK 1
  
```

Fig. 13. Arduino Code: The Initialization of Variables and Components

```

NavarroRGC_FinalProject_CPE162P.ino
51 int gradeBCount = 0;
52
53 void setup() {
54   Serial.begin(9600);
55   // Init serial port for DFPlayer Mini
56   softwareSerial.begin(9600);
57   lcd.init();
58   lcd.backlight();
59   lcd2.init();
60   lcd2.backlight();
61
62   pinMode(ENCODER_DT, INPUT);
63   pinMode(ENCODER_CLK, INPUT);
64
65   LoadCell.begin();
66   LoadCell.start(2000);
67   LoadCell.setCalFactor(calibrationValue);
68
69
70   lcd.clear();
71   lcd.setCursor(0, 0);
72   lcd.print("System Ready");
73   delay(500);
74
75   lcd2.clear();
76   lcd2.setCursor(3, 0);
77   lcd2.print("Good Day!");
78   delay(500);
79
80   displayHomeScreen();
81
82   // Start communication with DFPlayer Mini

```

Fig. 14. Arduino Code: Setup Function

```

NavarroRGC_FinalProject_CPE162P.ino
96
97 void loop() {
98   handleEncoder();
99
100   if (isAutoMode) {
101     if (!autoModeStarted) {
102       autoStartTime = millis(); // Record start time
103       autoDurationMinutes = 0.0; // Reset duration
104       autoModeStarted = true;
105     }
106     runAutoMode();
107   } else {
108     if (autoModeStarted) {
109       // Calculate duration in minutes (with decimals)
110       autoDurationMinutes = (millis() - autoStartTime) / 60000.0;
111
112       // Send only time duration to ESP32 (serial)
113       Serial.print("duration:");
114       Serial.println(autoDurationMinutes, 2);
115
116       autoModeStarted = false; // Reset flag
117     }
118
119     displayHomeScreen();
120   }
121
122   // Always update LCD2
123   lcd2.setCursor(0, 0);
124   lcd2.print("Grade A: ");
125   lcd2.print(gradeACount);
126   lcd2.print(" ");
127

```

Fig. 15. Arduino Code: Loop Function

For Fig. 16-18, the code connects an ESP32 to a Wi-Fi network and listens for incoming serial data from another device through pins 16 (RX) and 17 (TX). When it receives data, it checks if it's labeled with "data:" or "duration:" and formats it accordingly. The formatted data is then sent to a ThingSpeak channel using HTTP requests, allowing real-time cloud monitoring. This setup enables easy tracking of system metrics like duration through the internet.

```

ESP32_Connectino
1 #include <WiFi.h>
2 #include <HTTPClient.h>
3
4 const char* ssid = "PLDTHOMEFIBRT6ucX";
5 const char* password = "PLDTWIFITLK72";
6 const char* apiKey = "18A6ZBU4I3OLZ244";
7 const int channelID = 2964917;
8
9 #define RXD2 16
10 #define TXD2 17
11
12 String receivedData = "";
13
14 // Function declaration before usage
15 void sendToThingSpeak(String data);
16
17 void setup() {
18   Serial.begin(115200);
19   Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
20
21   WiFi.begin(ssid, password);
22   Serial.print("connecting to WiFi");
23   while (WiFi.status() != WL_CONNECTED) {
24     delay(500);
25     Serial.print(".");
26   }
27   Serial.println("\nWiFi connected");

```

Fig. 16. ESP32 Code: The Initialization of Variables

```

ESP32_Connectino
12
13 String receivedData = "";
14
15 // Function declaration before usage
16 void sendToThingSpeak(String data);
17
18 void setup() {
19   Serial.begin(115200);
20   Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
21
22   WiFi.begin(ssid, password);
23   Serial.print("connecting to WiFi");
24   while (WiFi.status() != WL_CONNECTED) {
25     delay(500);
26     Serial.print(".");
27   }
28   Serial.println("\nWiFi connected");

```

Fig. 17. ESP32 Code: Setup Function

```

ESP32_Connectino
25 Serial.print(".");
26 }
27 Serial.println("\nWiFi connected");
28 }
29
30 void loop() {
31   if (Serial2.available()) {
32     receivedData = Serial2.readStringUntil('\n');
33     receivedData.trim();
34
35     if (receivedData.length() > 0) {
36       // Check the prefix of the received data
37       if (receivedData.startsWith("data:")) {
38         // Future implementation for egg grade data (not yet handled)
39         String data = receivedData.substring(5);
40         // Example: sendToThingSpeak("field1=grade&field2=gradeB"); (not implemented here)
41
42       } else if (receivedData.startsWith("duration:")) {
43         String durationStr = receivedData.substring(9);
44         float minutes = durationStr.toFloat();
45
46         // Send to ThingSpeak as field only
47         String url = "fields=" + String(minutes, 2);
48         sendToThingSpeak(url);
49       } else {
50

```

Fig. 18. ESP32 Code: Loop Function

III. RESULTS AND DISCUSSION

The prototype of the smart egg grading system was successfully developed and tested, demonstrating its ability to sort eggs based on weight with the help of automation and IoT features. The hardware includes an Arduino Uno R3, which acts as the main controller, a load cell with an HX711 module to measure egg weight, servo motors for egg handling and sorting, a rotary encoder for mode selection, and a DFPlayer Mini for voice feedback. Two LCD screens were used to display

important data such as egg count, weight, sorting duration, and grading results. During testing, the system accurately sorted eggs into Grade A (28–55g) and Grade B (20–27g) categories. The voice feature provided clear audio prompts during each step of the grading process, helping users understand system activity. The ESP32 module successfully sent data to the ThingSpeak cloud platform, allowing remote monitoring of real-time data such as egg count and grading duration. The system also ran smoothly on a battery-powered setup using four 18650 lithium batteries, proving it to be a portable and cost-effective solution for small poultry farms that cannot afford commercial-grade equipment. Overall, the prototype met its goal of providing an affordable, user-friendly, and efficient egg grading system with cloud connectivity.

For Fig. 19-22, the ThingSpeak dashboard for the Smart Egg Grading System displays real-time data collected and sent by the IoT-based egg sorter. The system uses four fields: Field 1 shows the total number of eggs sorted, Field 2 shows the count of Grade A eggs, Field 3 displays the count of Grade B eggs, and Field 4 logs the system's operating duration. Each field has a chart to visualize changes over time and a numeric display for the most recent value. This setup allows remote monitoring of egg classification performance and usage patterns, making it easier to track productivity, performance, and trends through the ThingSpeak platform.

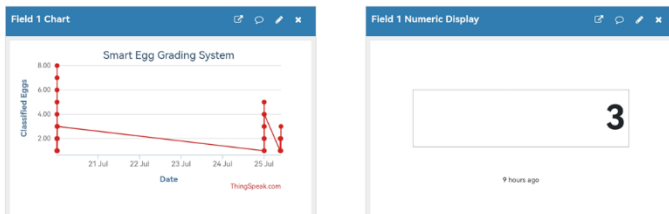


Fig. 19. ThingSpeak Visualization: Classified Egg Count

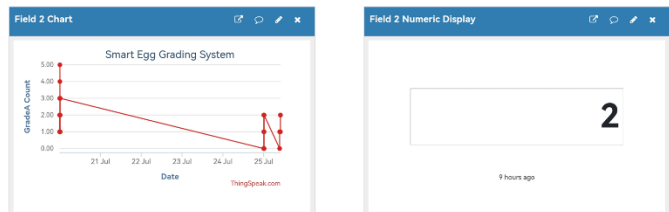


Fig. 20. ThingSpeak Visualization: Grade A Egg Count



Fig. 21. ThingSpeak Visualization: Grade B Egg Count

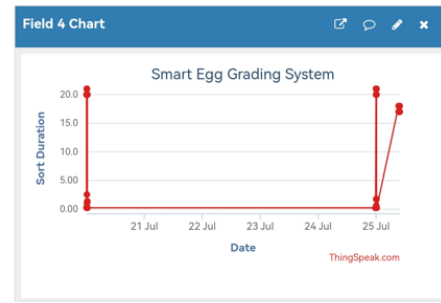


Fig. 22. ThingSpeak Visualization: Sorting Duration

IV. CONCLUSION AND RECOMMENDATION

The prototype of the smart egg grading system was able to successfully automate the egg sorting process using weight as the basis for grading. It effectively addressed the common problem of human error in manual grading and offered a low-cost alternative to expensive commercial machines. With the help of components like Arduino Uno R3, ESP32, load cell, servo motors, and LCDs, the system was able to sort eggs into two weight-based categories (Grade A and Grade B), display useful data on screens, give voice prompts, and send real-time updates to the cloud using ThingSpeak. It also allowed users to monitor the number of eggs sorted and the time taken, making it helpful for small poultry farm owners who want better tracking and organization.

For future improvements, it is recommended to upgrade the system to include more egg categories based on industry standards and integrate additional sensors (like crack or dirt detection) to improve accuracy. Adding a mobile app for easier access to cloud data can also make the system more user-friendly. Using stronger components or improving the mechanical design may help increase sorting speed and durability. Overall, the system is a promising and affordable solution that can benefit small farmers by reducing labor, improving accuracy, and providing better control over egg production.

REFERENCES

- [1] C. Rahmad, S. E. Sukmana and A. R. Syulistyo, "An Automatic Egg Quality Grading Using Nature-Inspired Algorithm Based Classification," 2022 5th International Conference on Information and Communications Technology (ICOIAC), Yogyakarta, Indonesia, 2022, pp. 194-198, doi: 10.1109/ICOIAC55506.2022.9972134.
- [2] "Egg Grades & Sizes | Overview & Scale | Study.com," study.com, 2023. <https://study.com/academy/lesson/market-classes-grades-of-eggs.html>
- [3] R. L. M. Bambilla, M. J. B. Lee and F. L. Valiente, "Freshness Condition Detection, Size Classification with Weight Prediction and Sortation of Philippine Egg Produce Using YOLOv8 and OpenCV," 2025 17th International Conference on Computer and Automation Engineering (ICCAE), Perth, Australia, 2025, pp. 158-164, doi: 10.1109/ICCAE64891.2025.10980499. keywords: {Industries;Computer vision;Accuracy;Costs;Correlation;Production;Manuals;Reliability engineering;Personnel;Standards;egg quality;egg freshness;egg weight;egg size;sortation;computer vision;YOLOv8;OpenCV},